SP-201 Red Multi-Agent Research & Fact-Checking Pipeline

FINAL REPORT

CS4850 | Section 2 | Spring 2026 April 24, 2026, Professor Perry
Website Link: https://pj201-senior-project-2026.github.io/ProjectWebsite
GitHub Link: https://github.com/PJ201-Senior-Project-2026/SP201-Red-MA-2026
Video Presentation: https://www.youtube.com/watch?v=RzLpxnUCO7U

STATS and STATUS

- **Lines of Code:**
  - Agent: 71
  - Tools: 64
  - Utils: 110
  - Entry point: 63
  - **Total: 308**
- **Project Components:** 3 (Agent, Tools, Utils)
- **Tools:** 3 (Search, View Search History, Save Search)
- **Total Hours:** 150
- **Status:** 100% Complete

1. Introduction

1.1 Overview

The goal of this project is to design and implement a multi-agent autonomous research and fact-checking system using large language models and agent orchestration frameworks. The system is built to process complicated, open-ended research prompts by breaking them down into structured subtasks and coordinating several specialized agents to generate a verified, credited research report.
A central Manager Agent evaluates the user prompt and assigns tasks to several worker agents in the architecture's Manager–Worker pattern. A Search Agent uses free, publicly available APIs to retrieve pertinent data from external data sources. A Synthesizer Agent incorporates the collected data into a logical draft, while a Fact-Checker Agent cross-references assertions with the original sources to identify discrepancies and reduce hallucinations.
The finished system demonstrates how autonomous agents can work together to enhance research reproducibility, transparency, and reliability in AI-assisted knowledge creation.

1.2 Project Scope

This project's scope includes:

- Structured prompt decomposition

- Multi-agent task orchestration
- Session state management
- External data retrieval via publicly accessible APIs
- Automated fact-checking of generated content

The project **does not** include:

- Enterprise-grade scaling capabilities
- Paid API integration
- Large-scale deployment

1.3 Definitions and Acronyms

- **Agent:** A software entity that operates independently to accomplish a given task
- **Manager Agent:** Responsible for assigning tasks to worker agents
- **Search Agent:** Acquires data from external resources
- **Synthesizer Agent:** Develops a well-structured research draft
- **Fact-Checker Agent:** Validates claims with sources found
- **LLM:** Large Language Model
- **ADK:** Agent Development Kit

1.4 Assumptions

- Prompts will be given by users in English
- Public, free-tier APIs (search and LLM) will remain accessible
- Users are technically literate enough to use a simple web interface or CLI
- A reliable internet connection is available during system operation

1.5 References

- Google ADK Documentation
- Google AI Studio Documentation

---

2. Requirements

2.1 Constraints

2.1.1 Latency and Timeouts The system supports long-running multi-agent workflows without premature HTTP timeouts. Structured API error handling and retry logic handle network instability, rate limiting, and transient API failures.
2.1.2 Token Limits The system imposes prompt size limits to guarantee compatibility with LLM context window requirements. Prompts exceeding predetermined token thresholds are truncated, summarized, or rejected by the Manager Agent.

2.2 Functional Requirements

2.2.1 Manager Agent

- Accepts a research prompt supplied by the user
- Breaks down the prompt into at least three well-structured sub-questions
- Assigns one or more instances of the Search Agent to each sub-question
- Organizes workflow flow for each agent
- Restarts search or synthesis if the Fact-Checker Agent rejects the draft
- Aggregates and returns the final approved output to the user

2.2.2 Search Agent

- Accesses external data sources through authorized public APIs
- Obtains pertinent textual information about designated sub-questions
- Provides organized search results with URLs and source titles
- Utilizes retry mechanisms to address API errors

2.2.3 Synthesizer Agent

- Combines search results into a logically organized, cohesive draft
- Preserves citation references for retrieved sources
- Creates output in markdown format for user interface display

2.2.4 Fact-Checker Agent

- Compares synthesized statements with retrieved original content
- Determines which claims are unsubstantiated or unverifiable
- Flags or rejects inconsistent drafts
- Initiates re-execution of Search or Synthesis phases when required

2.2.5 Session and State Management

- Preserves session state throughout multi-agent execution
- Tracks stages: pending, decomposing, searching, synthesizing, fact-checking, finalizing
- Keeps track of intermediate outputs for debugging and traceability

2.3 Non-Functional Requirements

2.3.1 Security

- No permanently identifiable user data stored
- API keys and credentials stored safely using environment variables

2.3.2 Capacity

- Must support a minimum of one concurrent active session during development
- Architecture must be flexible for future multi-user scaling

2.3.3 Usability

- Offers a basic web-based interface or CLI
- Real-time workflow status updates displayed on the interface
- Clickable citations included in the finished product

2.3.4 Other

- All factual statements in the final report must have verifiable citations
- Workflow auditable through visible state transitions

2.4 External Interface Requirements

2.4.1 User Interface

- Prompt input field
- Real-time agent activity status indicator
- Structured markdown output section
- Clickable citation links

2.4.2 APIs

- DuckDuckGo API (or comparable public search API) for data retrieval
- Gemini API for large language model analysis
- Optional MySQL database for search result storage

2.4.3 Database/Protocol

- All API communication over HTTPS
- REST-based communication protocols supported

---

3. Analysis and Design

3.1 Purpose of the SDD

This System Design Document (SDD) provides an architectural and technical description of the SP-201 Red Multi-Agent Research & Fact-Checking Pipeline. It translates the Software Requirements Specification (SRS)'s functional and non-functional requirements into a structured system architecture and component-level design.

3.2 Design Considerations

3.2.1 Assumptions and Dependencies

- Python 3.10 or later
- Google ADK for agent orchestration
- Gemini API as the major LLM provider

- Public search API (DuckDuckGo) accessible
- English-language prompts
- Single-user setting during development
- Internet connection required
- Future extensions may include additional agents (bias detection, citation formatting)

3.2.2 General Constraints

- LLM token and context window limits
- External API rate limits
- Performance restrictions (typically less than a minute for ordinary prompts)
- Environment variable security requirements
- HTTPS for all external communications
- Repeatable execution processes for validation

3.2.3 Goals and Guidelines

- Modular division of agent responsibilities
- Reasoning transparency and citation traceability
- Reduction of hallucinated outputs
- Clear workflow state transitions
- Flexibility for future agent additions
- Correctness and verification prioritized over raw performance speed

3.2.4 Development Methods Progressive and modular design process. Each agent component is independently developed and evaluated before integration. Agents are organized as discrete modules with well-defined interfaces using an object-oriented methodology.

3.3 Architectural Strategies

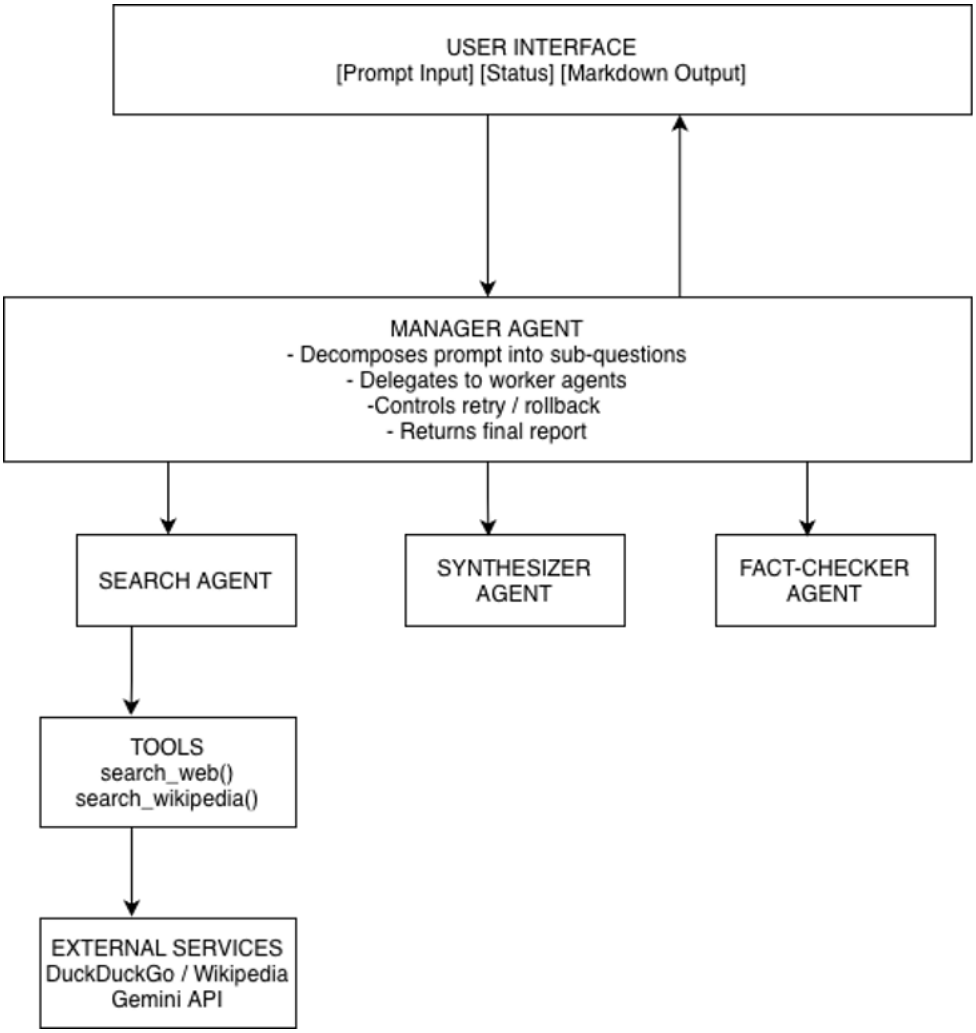
Strategy 1: Hierarchical Orchestration A central Manager Agent manages workflow progression. Worker agents (Search, Synthesizer, Fact-Checker) function independently but are coordinated through structured message forwarding. *Alternative rejected:* Monolithic single-agent design — poor modularity, less verification control, decreased transparency.
Strategy 2: Sequential Execution Model Agents work in regulated, sequential stages: Pending → Decomposing → Searching → Synthesizing → Fact-Checking → Finalizing *Alternative rejected:* Parallel uncontrolled execution — increased synchronization difficulty and inconsistent state risk.
Strategy 3: Modular Agent Components Each agent is implemented as a stand-alone software module with distinct input/output schemas, enabling:

- Self-directed testing
- Future agent replacement
- Scalability for additional agents

3.4 System Architecture

The system is broken down into the following high-level subsystems:

1. User Interface Subsystem
2. Manager Agent Subsystem
3. Search Agent Subsystem
4. Synthesizer Agent Subsystem
5. Fact-Checker Agent Subsystem
6. Session State Management Subsystem
7. External API Integration Subsystem



Agent Coordination Flow: User → Manager Agent → Search Agent → Manager Agent → Synthesizer Agent → Fact-Checker Agent → Manager Agent → User
If validation fails, control returns to the Search or Synthesizer stage.

3.5 Detailed Component Design

3.5.1 Manager Agent

- **Classification:** Application-level orchestration module
- **Responsibilities:** Accept prompts, decompose into sub-questions, delegate tasks, manage state transitions, handle rollback, aggregate output
- **Constraints:** Must operate within LLM token limits, maintain valid state transitions, prevent infinite validation loops
- **Interface:** `process_prompt(prompt: string) → final_report`, `get_workflow_status() → state_identifier`

3.5.2 Search Agent

- **Classification:** External data retrieval module
- **Responsibilities:** Submit queries, retrieve textual content, extract source titles/URLs, return structured results
- **Constraints:** Subject to API rate limits, network availability, external API schema
- **Interface:** `search(query: string) → structured_results`

3.5.3 Synthesizer Agent

- **Classification:** Content generation module
- **Responsibilities:** Organize data by topic, generate structured paragraphs, preserve citations, format in markdown
- **Constraints:** Must operate within LLM token limits, must not fabricate unsupported claims
- **Interface:** `synthesize(search_results) → draft_report`

3.5.4 Fact-Checker Agent

- **Classification:** Validation and verification module
- **Responsibilities:** Compare claims to sources, identify unsupported statements, return validation status, trigger reprocessing
- **Constraints:** Limited to retrieved data, dependent on LLM evaluation accuracy
- **Interface:** `validate(draft, sources) → validation_status`

3.5.5 Session State Manager

- **Classification:** State tracking and workflow control module
- **Responsibilities:** Track execution stage, store intermediate outputs, provide traceability
- **Interface:** `update_state(new_state)`, `get_state()`

---

4. Development

4.1 Implementation Detail Narrative

The SP-201 Red Multi-Agent Research & Fact-Checking Pipeline is implemented as a hierarchical Manager–Worker multi-agent system, consistent with architectural decisions documented in the SRS and SDD.
The system integrates six primary components: User Interface (CLI or web-based), Manager Agent, Search Agent, Synthesizer Agent, Fact-Checker Agent, and Session State Manager.

Communication follows a structured, sequential message-passing model. All sub-agents are treated as tools rather than individual specialists, so orchestration goes from agent to agent to ensure the workflow pipeline is not interrupted. All communication occurs through structured data objects (JSON-like schemas). Sequential execution is enforced to prevent race conditions and maintain deterministic state transitions.
External Technologies Integrated:

- Python 3.10+
- Google ADK for agent orchestration
- Gemini API (Free Tier models such as gemini-1.5-flash)
- Public search APIs for external data retrieval

4.2 System Integration & Workflow Overview

Operational Workflow:

1. User submits a complex research question through the CLI interface
2. Manager Agent analyzes and decomposes the prompt into at least three structured sub-questions
3. Each sub-question is submitted to the Search Agent
4. Search Agent calls an external search API, parses returned JSON, extracts text and citation metadata, returns structured results
5. Manager Agent aggregates search results and forwards them to the Synthesizer Agent
6. Synthesizer Agent constructs a structured prompt and uses Gemini to generate a cohesive markdown-formatted draft
7. Draft and source data are passed to the Fact-Checker Agent
8. Fact-Checker compares generated claims against retrieved source material
9. If unsupported statements are found, Manager reinitiates Search or Synthesis phase
10. If validation succeeds, the final research report with clickable citations is displayed to the user

Deterministic State Sequence: Pending → Decomposing → Searching → Synthesizing → Fact-Checking → Finalizing

4.3 Global Design Decisions

- **Manager–Worker Architecture:** Centralized orchestration for improved modularity, traceability, and verification control
- **Sequential Execution Model:** Prevents race conditions and simplifies debugging
- **Structured JSON Data Objects:** Ensures consistency and citation traceability between agents
- **Global Error Handling Strategy:** Structured exception handling and retry logic for transient failures and rate limits
- **Token Management Controls:** Prompt size limits and truncation safeguards for free-tier Gemini models
- **Environment Variable Security:** API keys stored in environment variables, never hard-coded

4.4 Component Development

4.4.1 Manager Agent Implemented as a Python module using Google ADK:

- Prompt decomposition logic using Gemini

- Structured schemas for sub-question generation
- Deterministic state machine for execution phases
- Rollback logic for failed validation loops
- Controlled sequential integration with worker agents

4.4.2 Search Agent Implemented using a FunctionTool integration within Google ADK:

- API wrapper for public search endpoints (DuckDuckGo)
- JSON response parser
- Content, source title, and URL extraction
- Citation-preserving result objects
- Retry logic for rate limits and API failures

4.4.3 Synthesizer Agent Implemented using Gemini via Google ADK:

- Structured prompt construction using aggregated results
- Token usage controls for free-tier constraints
- Markdown formatting with inline citation markers
- Safeguards against unsupported claim generation

4.4.4 Fact-Checker Agent Developed using Gemini-based evaluation prompts:

- Factual claim extraction from drafts
- Comparison against original retrieved content
- Unsupported statement flagging
- Validation status returned to Manager Agent
- Controlled re-execution triggering

4.4.5 Memory Service The Memory Service enables retention and recall of information from past conversations. Without memory, each session starts with no prior context. Implementation used Google ADK's built-in `InMemoryMemoryService`:

- Imported the `load_memory` tool for searching past conversations
- Implemented an `after_agent_callback` function to save completed sessions
- Added instructions for the Manager Agent to use `load_memory` when past context is relevant
- Configured `InMemoryMemoryService` as the default memory backend

Known Memory Limitations:

- Not persistent across server restarts (RAM-only)
- Search limited to basic keyword matching
- `VertexAiMemoryBankService` recommended for production persistence and semantic search

4.5 Database and External Connections

4.5.1 Connection Overview The system primarily connects to external data sources rather than a traditional relational database:

- Gemini API (Google AI Studio Free Tier)
- DuckDuckGo (or similar public search API)

A MySQL database may optionally be used for session logs or search results, but is not required for core functionality.

4.5.2 Configuration Details Required environment variables:

- `GEMINI_API_KEY` – Google AI Studio API key
- `SEARCH_API_ENDPOINT` – Search API base URL
- `SEARCH_API_KEY` (if required)

Example:

```
export GEMINI_API_KEY=your_api_key_here
export SEARCH_API_ENDPOINT=https://api.duckduckgo.com/
```

All communication occurs over HTTPS.

4.5.3 Integration Logic

- Search Agent submits GET requests for JSON search results
- Synthesizer and Fact-Checker submit prompts to Gemini via POST
- Responses parsed, validated, and transformed into structured internal objects
- All external communication wrapped in structured error-handling blocks

4.6 Project Setup and Environment Configuration

4.6.1 Prerequisites

- Software: VS Code / Python 3.10, Git, Google ADK, Microsoft Access 2016, PIP
- Hardware: Internet access for Google API

4.6.2 Installation Steps

1. Clone the repository:
2. `git clone https://github.com/PJ201-Senior-Project-2026/agent_server.git`
3. Install dependencies:
4. `pip install -r requirements.txt`
5. Configure environment variables — create a `.env` file based on `.env.example` and add `GOOGLE_API_KEY`

4.6.3 Verification Correct setup is confirmed when:

- CLI prompts for a research query
- Workflow stages print in sequence (Decomposing → Searching → Synthesizing → Fact-Checking → Finalizing)
- A formatted markdown report with citations is displayed
- Misconfigured APIs return a structured error message

4.7 Development Summary

The current build successfully implements hierarchical multi-agent coordination, external data retrieval, structured synthesis, and automated fact-checking. The system operates within free-tier API constraints and supports a full end-to-end research workflow.

---

5. Testing

5.1 Test Plan

The system was tested at both the **component level** and the **system level** to ensure correct functionality, reliability, and robustness across the full multi-agent workflow.

Testing focused on the following areas:

- **End-to-End Workflow Testing**
  Verified that a complete user prompt successfully flows through all system stages:
  Decomposing → Searching → Synthesizing → Fact-Checking → Finalizing
  Ensured that the final output is generated correctly with structured formatting and citations.
- **Agent-Level Functional Testing**
  Each agent was tested independently:
  - Manager Agent: Prompt decomposition and task delegation
  - Search Agent: API query execution and structured result formatting
  - Synthesizer Agent: Coherent report generation with citations
  - Fact-Checker Agent: Detection of unsupported claims
- **API Communication Testing**
  Verified successful interaction with:
  - Gemini API (LLM responses)
  - DuckDuckGo / Search API (data retrieval)
    Tested handling of:
  - Invalid API keys
  - Rate limits
  - Empty or malformed responses
- **State Transition Testing**
  Confirmed that the system follows the correct deterministic state sequence and does not skip or repeat invalid states.
  Ensured rollback behavior functions correctly when validation fails.

- **Error Handling and Retry Logic Testing**
  Simulated failures such as:
  - API timeouts
  - Network issues
  - Incomplete search results
    Verified that retry logic executes correctly and prevents system crashes.
- **Memory Functionality Testing**
  Tested that:
  - Sessions are automatically saved after execution
  - Past queries can be retrieved using keyword search
  - Memory integrates correctly with the Manager Agent

-

5.2 Test Results

The system consistently completed the full pipeline and produced structured markdown reports with citations.

- **Agent Coordination**
  All agents operated correctly within the Manager–Worker architecture, with proper task delegation and sequencing.
- **Fact-Checking Effectiveness**
  The Fact-Checker Agent successfully identified unsupported claims and triggered reprocessing when necessary, reducing hallucinated outputs.
- **State Management**
  State transitions were deterministic and traceable, ensuring predictable system behavior.
- **Error Handling**
  Retry mechanisms handled transient API failures effectively, preventing system crashes and ensuring continuity.
- **Memory Integration**
  Memory successfully stored and retrieved session data within runtime, improving efficiency for repeated or related queries.

5.3 Limitations of Testing

- Testing was conducted in a **single-user environment**
- Memory testing was limited to **in-memory storage (non-persistent)**
- External API responses may vary, affecting reproducibility
- Fact-checking accuracy depends on quality of retrieved sources and LLM evaluation

6. Version Control

The project uses **Git and GitHub** for source code management and team collaboration. All code, documentation, and project artifacts are version-controlled.
Version control is important because it enables collaborative development, tracks project history, allows rollback to prior stable versions, supports parallel feature development through branching, and provides accountability through commit attribution.

6.1 Repository Structure

Repository: https://github.com/PJ201-Senior-Project-2026/SP201-Red-MA-2026

6.2 Commit History

The project includes a comprehensive commit history tracking:

- Initial project setup and environment configuration
- Core agent application development
- Frontend/CLI interface implementation
- Package integration and tool visualization
- Package explanation implementation
- Error handling and data validation
- Testing and bug fixes
- Documentation updates

---

7. Conclusion

The system successfully demonstrates a multi-agent pipeline for autonomous research and fact-checking. It shows how coordinated autonomous agents can improve the quality and legitimacy of AI-generated research results through modular orchestration, structured data flow, and validation-driven workflow control.

7.1 Limitations

- Dependency on external API availability
- Free-tier rate limits and token constraints
- Sequential execution may increase latency for complex prompts
- Validation accuracy depends on retrieved source quality and LLM evaluation

7.2 Future Work

- Parallel agent execution
- Persistent memory (migration to `VertexAiMemoryBankService`)
- Result caching
- Improved UI visualization
- Additional validation agents (bias detection, citation formatting)

---

8. Appendix

8.1 Tools and Technologies

- Python 3.10
- Google ADK
- Gemini API (gemini-1.5-flash)
- DuckDuckGo Search API
- Git / GitHub

8.2 Glossary

- **Agent:** Autonomous software component performing a defined task
- **Manager–Worker Pattern:** Hierarchical orchestration design pattern
- **LLM:** Large Language Model
- **Hallucination:** Unsupported or fabricated content generated by an LLM
- **Session State:** Stored execution context across agent stages

8.3 Bibliography

- Google ADK Documentation
- Google AI Studio Documentation
- Multi-Agent Systems Design Literature
- Software Engineering Design Pattern References